Vision Transformers

Applied Deep Learning — Chapter 17

Yin Yang

Foundation Books Project

Roadmap

Running Example and Applied Goal

Patch Tokens Set the Evidence Budget

An image model begins with a decision: what evidence is the model allowed to see?

- A ViT splits the image into patch tokens, then applies a Transformer to the resulting sequence.
- A coarse grid blurs crisp edges, garnishes, sauce boundaries, or small printed marks before attention can use them.
- A finer grid keeps detail but produces many more tokens.

Attention can connect tokens; it cannot restore detail that never became a token.

Running Task: Food-101

Map photographs to dish labels. Some classes need:

- local texture (crisp edges, sauce patterns),
- broader layout (container shape, garnish placement),
- relationships between several food regions.

Detail-sensitive images (charts, screenshots, rendered math) appear as stress tests — thin axis labels, decimals, operators can vanish under resize or fall awkwardly across patch boundaries.

What the Recipe Must Report

A complete pretrained-ViT run names:

- data source, split rule,
- image preprocessing, patch size,
- model checkpoint, trainable layers,
- optimizer, learning rate, batch size,
- precision, hardware,
- stopping rule, evaluation metric.

Accuracy, elapsed time, throughput, peak memory are measurements of the *same experiment*, not separate afterthoughts.

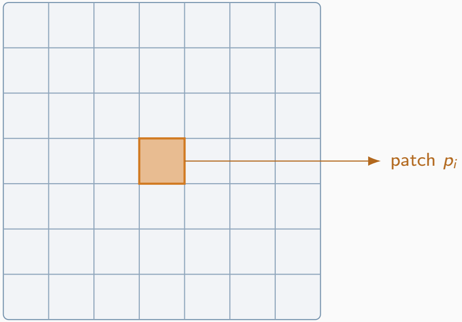
From Pixels to Patch Tokens

Patches and Patchification

Patch
A spatial block of $x \in \mathbb{R}^{H \times W \times C}$. A non-overlapping $P \times P$ patch holds P^2C values.

$$N = \frac{H}{P} \cdot \frac{W}{P}.$$

224×224 RGB, $P = 16$: $14 \cdot 14 = 196$ patches, each $16^2 \cdot 3 = 768$ numbers.



$P \times P$ tile

Patch Embedding: Linear Map

Flattened patch $p_i \in \mathbb{R}^{P^2 C}$, learned map:

$$e_i = p_i W_E + b_E, \quad e_i \in \mathbb{R}^d, \; W_E \in \mathbb{R}^{P^2 C \times d}.$$

224×224 image, $P = 16$, $d = 768$:

$$W_E \text{ has } 768 \cdot 768 = 589{,}824 \text{ weights.}$$

Patch size becomes an information choice. A tiny mark, decimal, or thin operator can be averaged into a patch before attention sees it.

Tiny Patch-Embedding Calculation

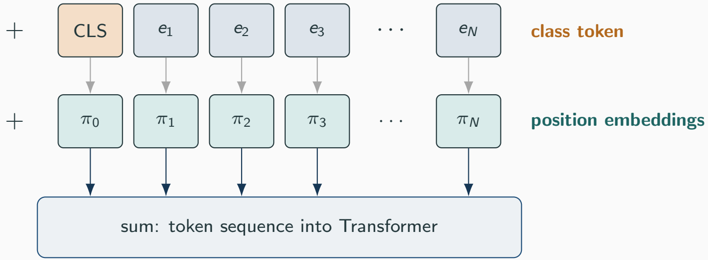
4×4 grayscale image; upper-left 2×2 patch flattens to $p_1 = [1, 0, 0, 1]$. Two-dimensional embedding:

$$W_E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 0 \end{bmatrix}, \quad b_E = [0.5, -0.5].$$

$$p_1 W_E + b_E = [1, 0, 0, 1] W_E + [0.5, -0.5] = [0, 0] + [0.5, -0.5] = [0.5, -0.5].$$

Same idea as a real ViT, just larger numbers.

Class Token and Position Information



Without position info, top-left and bottom-right look identical. CLS state at the output feeds the classifier head.

Resolution Changes the Token Count

384×384 RGB with 16×16 patches:

$$\frac{384}{16} \cdot \frac{384}{16} = 24 \cdot 24 = 576 \text{ tokens.}$$

Raw patch length unchanged at 768.

- Tokens went from $196 \rightarrow 576$ (about $3\times$).
- Higher resolution preserves small evidence *and* multiplies attention work.
- Resolution is not a free knob.

Self-Attention Over Image Patches

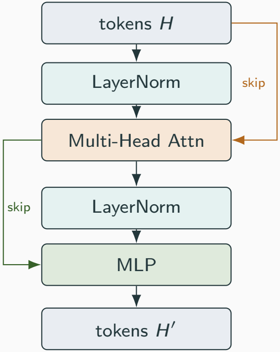
Self-Attention with Image Tokens

Token matrix $H \in \mathbb{R}^{N \times d}$. For one head:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V.$$

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right), \quad \mathrm{Attention}(H) = AV.$$

Same formula as text self-attention — N comes from image resolution and patch size. Each head produces an $N \times N$ score matrix.



one ViT block

Attention Cost Grows Quadratically with Tokens

Per-head score count is N^2.

Image	Tokens N	Scores N^2
$224 \times 224,\ P = 16$	196	38,416
$384 \times 384,\ P = 16$	576	331,776
$224 \times 224,\ P = 8$	784	614,656

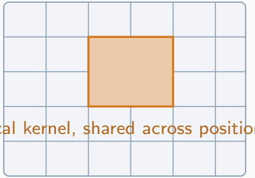
Side length $\times 1.7 \Rightarrow$ scores $\times 8.6$. Patch size halved $\Rightarrow$ scores $\times 16$. Cost follows tokens.

Multi-head multiplies work by a constant factor; it does not remove the issue.

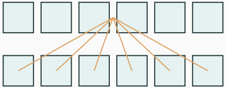
ViTs vs. CNNs

Different Starting Assumptions

CNN



ViT



one token attends to many

CNN

- Local-pattern bias, parameter sharing.
- Receptive field grows with depth.

ViT

- Broad interactions from layer one.
- Less built-in knowledge nearby pixels relate.

Plain ViT often needs a strong checkpoint; DeiT showed careful training closes the gap.

Pretrained Baselines as First Experiments

Starting point	Useful when	Applied warning
CNN baseline	small data, speed matters, local texture is important	may miss global layout
Pretrained ViT	transfer checkpoint exists, global evidence matters	token count makes high resolution expensive
Hybrid / modern ConvNet	fast training, Transformer-era recipes	still needs validation discipline
Hierarchical Transformer	high-res or multi-scale evidence	cost must be measured at chosen input size

Baseline does not mean weak. A pretrained ResNet, modern ConvNet, DeiT, or ViT is a serious first opponent.

Mistake Galleries Decide Next Experiments

Collect a small set of validation examples where the model is wrong, with input, true label, predicted label, confidence.

- Visually similar food classes confused $\Rightarrow$ stronger checkpoint or augmentation.
- Tiny printed labels missed in a chart $\Rightarrow$ image resolution or document model.
- Mistakes on *corrupted labels* $\Rightarrow$ no model change will help.

Start with measurements and examples, not the assumption that the newest architecture must be best.

Training ViTs in Practice

The Boring First Run Is on Purpose

Realistic starting point: a pretrained checkpoint.

1. Load a small pretrained ViT or DeiT-style checkpoint.
2. Resize images to checkpoint's expected size.
3. Replace the classification head.
4. Start with little or none of the encoder changing.
5. Inspect wrong examples *before* touching five knobs.

Blurry labels $\Rightarrow$ resolution or preprocessing. Repeated fine-grained confusions $\Rightarrow$ data, augmentation, or stronger checkpoint.

Trainable Parameters: Three Choices

Pretrained encoder $\approx$ 85M parameters; new head 77,669 parameters.

Frozen body update only the head ($< 0.1\%$ of model). Fast, stable, may underfit on shifted images.

Full fine-tune update everything. Most adaptation capacity, most memory.

LoRA small low-rank updates on selected linear layers (often attention W_Q, W_V); pretrained weights frozen. Middle path: more parameters than head-only, far fewer than full fine-tune.

Position Embeddings and Resolution Transfer

Patch embedding weights reuse easily when patch size and channel count stay fixed — but the *number of patch positions* changes. 224×224 with $P = 16$: $14 \times 14 = 196$ positions.

384×384 with $P = 16$: $24 \times 24 = 576$ positions.

Position-embedding interpolation
Treat the 14×14 learned position grid like a small feature map and interpolate to 24×24, then flatten. Class-token position is kept separate.

Resolution transfer has a shape cost. Higher resolution increases tokens, activation memory, attention cost.

Training-Knob Cheat Sheet

Choice	Typical effect	Applied warning
larger image	preserves small details	increases tokens / cost
smaller patch	finer evidence	attention cost rises sharply
larger batch	better throughput, stability	may exceed memory
mixed precision	faster, less memory	check numerical issues
freezing layers	saves memory, less overfit	may limit adaptation
strong augmentation	better generalization	can damage labels (flip a chart, crop a tiny exponent)

Image / patch size multiply the activations kept during forward–backward. Familiar knobs change memory and throughput dramatically.

Quality and Cost Together

Two runs reach the same validation accuracy.

- Run A: 224×224, 18 min, peak 7 GB.
- Run B: 384×384, 63 min, peak 15 GB.

Similar mistakes $\Rightarrow$ Run A is the better recipe.
Run B reads small labels Run A misses $\Rightarrow$ extra cost may be justified. Conclusion depends on measured errors, not on resolution alone.

Efficient ViTs: The Swin Transformer

Window Attention

Global attention pays for every token-to-token comparison.

Window attention
Self-attention applied separately inside local groups of tokens.

$N = 1024$ tokens, full attention: $1024^2 = 1{,}048{,}576$ scores per head. 16 windows of 64 tokens each:

$$16 \cdot 64^2 = 65{,}536 \text{ scores per head.}$$

$16\times$ fewer scores in that layer.

Shifted Windows and Hierarchical Features

Tradeoff of pure window attention: a token cannot see every other token immediately.

- Shifted windows let information cross window boundaries across layers.
- Multiple layers stack the effect.

Hierarchical representation
Features at multiple spatial resolutions: early stages have many fine-grained tokens; later stages merge neighbors into fewer tokens with wider feature vectors. Like a CNN becoming coarser and more semantic.

Useful for dense tasks (detection, segmentation later).

When Efficient Attention Earns Its Keep

Justify by measurement, not architecture fashion.

- Higher-res ViT: $+$ accuracy, $-$ memory, $-$ throughput. Show the tradeoff.
- Mistake gallery shows missed tiny details $\Rightarrow$ smaller patch or larger input may be worth it.
- Errors are ordinary class confusions $\Rightarrow$ better augmentation, fine-tuning, or stronger checkpoint may matter more.

Efficient ViTs are tools to make the desired evidence affordable; they do not remove the need for a baseline, ablation, and cost report.

Self-Supervised Visual Pretraining

Why Pretraining Matters Even Without Pretraining

Most applied projects never pretrain a ViT from scratch — but the downloaded checkpoint owes its usefulness to its pretraining method.

- Pretext task: a constructed task whose main purpose is to learn representations for later use.
- Target is built from the data, not manual labels.
- Representation transfer reuses features from one task for another.

Masked image modeling
Hide part of an image; train the model to predict missing pixels, representations, or patch content from the visible part.

196 patches, mask 75%: 147 hidden, 49 visible. Pixel-reconstruction loss (MAE):

$$\mathcal{L}_{\mathrm{MIM}} = \frac{1}{|M|} \sum_{j \in M} \|x_j - \hat{x}_j\|_2^2.$$

Other variants reconstruct latent features or discrete targets.

DINO and DINOv2: Self-Distillation Across Views

Two augmented views of the same image $\rightarrow$ learner network and teacher network.

- Learner trained to match teacher's distribution for another view.
- Teacher is a slow-moving copy of the learner (e.g. moving average), *not* an independent supervised model.
- No human class labels required.

Training signal: different views of the same image should give stable visual representations.
DINOv2 pushes this to stronger general-purpose visual checkpoints.

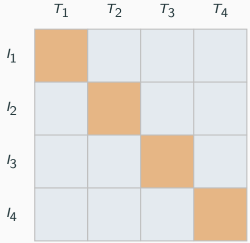
CLIP-Style Image-Text Pretraining

Paired images and text. Image-to-text loss for batch B:

$$\mathcal{L}_{\text{i2t}} = -\frac{1}{B}\sum_i \log \frac{\exp(z_i^\top t_i/\tau)}{\sum_j \exp(z_i^\top t_j/\tau)}.$$

Symmetric text-to-image term. Matched caption on the diagonal; other captions are contrastive negatives.

Useful zero-shot baseline when class names are prompts: `a photo of ramen`, `a photo of chocolate cake`.



diagonal = matched pairs

Pretraining Types at a Glance

Pretraining type	Target signal	Transfer intuition
supervised classification	human class labels	features for named categories
contrastive image-text	image / text pairs	align visual and language representations
masked image modeling	hidden patch content	learn structure from incomplete images
self-distillation (DINO)	teacher outputs across views	stable features without manual labels

Transfer must earn trust. A strong checkpoint is a hypothesis; target validation examples decide whether it transfers.

Evaluation, Reporting, Debugging

The Compact ViT Report

Earlier supervised-training checklist plus ViT-specific fields:

- checkpoint name,
- image size, patch size,
- trainable parameter count,
- adapter or full-fine-tune choice,
- gradient accumulation, precision,
- throughput, peak memory,
- several inspected mistakes.

Without it, a prettier validation number can hide a changed experiment.

Error Analysis Drives Next Change

Validation set of 600 images, 510 correct $\Rightarrow$ accuracy 85%. Inspect the 90 mistakes:

- Mostly low-resolution with small text $\Rightarrow$ increase image size.
- Visually similar classes $\Rightarrow$ data augmentation or stronger checkpoint.
- Mislabeled examples $\Rightarrow$ split deserves attention before the model does.

Ten inspected failures are often enough to change the next experiment.

Frozen Body vs. LoRA Adapters (Food-101 Subset)

Both rows: `google/vit-base-patch16-224-in21k`, 224×224, same training subset, same 600-example validation split.

Run	Val acc.	Time	Examples/s	Peak mem.
Frozen body	72.5%	33.7 s	510.6	0.433 GB
LoRA adapters	79.5%	63.1 s	250.0	1.387 GB

LoRA: $+7$ pp accuracy, but roughly $2\times$ time, half throughput, more memory. Trained 667,493 parameters vs. 77,669 for the head only. Mistake table still confuses fine-grained pairs (tuna tartare vs. beef tartare; miso soup vs. ramen).

Leakage Is Subtle in Image Work

Ways validation can lie:

- Duplicate images across splits.
- One source image cropped into multiple examples that land in both train and test.
- Test set inspected repeatedly while hyperparameters are chosen.

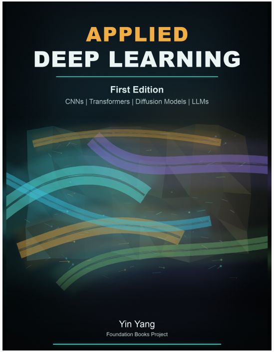
These are not small administrative errors. They change what the reported number means.

Summary

Summary

- ViT $=$ image as a sequence of patch tokens; Transformer mixes information across the sequence.
- Resolution and patch size set token count N; global attention cost grows like N^2.
- Pretrained checkpoint, image size, trainable layers, batch, precision, augmentation, time, throughput, memory all shape the result.
- Resolution transfer needs position-embedding interpolation and a new cost measurement.
- Swin: window attention, shifted windows, hierarchical features reduce cost while keeping spatial structure.
- Self-supervised, image-text, and self-distillation pretraining give different reasons a checkpoint may transfer.
- Validation examples decide. Test set for the final claim.

Next: CNNs revisited, vision-language models, object detection, segmentation — same evidence discipline, different output contracts.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/applied-deep-learning`
Code	`github.com/foundation-books/applied-deep-learning-code`
Print	`amazon.com/dp/B0GZF7QRSH`
Kindle	`amazon.com/dp/B0GZF9221X`
License	CC BY-NC-ND 4.0